**International Journal of Computer Applications (IJCA)**
Volume 4, Issue 1, January-December 2023, pp. 1-8, Article ID: IJCA_04_01_001
Available online at https://iaeme.com/Home/issue/IJCA?Volume=4&Issue=1
Journal ID: 5716-6932
© **IAEME** Publication

# STREAMLINING AWS SERVERLESS DEPLOYMENTS: A BITBUCKET PIPELINE APPROACH

**Sai Teja Makani**
Department of Electronics and Computer Engineering,
K L University, Green Fields, Vaddeswaram, Guntur-522302, India

## ABSTRACT

*The success of many companies hinges on the timely deployment of their products to the market. Even if a product offers exceptional utility, its value diminishes if its availability falls short of expectations. An efficient deployment approach is crucial to facilitate the transition of innovations into the market. This principle extends to software applications, where delayed deployment can result in user disinterest or shifting market dynamics. In today's fast-paced business landscape, characterized by rapid changes in both corporate and governmental spheres, uncertainty in the software field is ubiquitous. The ability to swiftly adapt and deploy changes has become paramount for success. Deployment processes have been integral to various industries throughout history. In ancient times, ship construction followed specific steps to transport vessels to the seashore, a process known as a slipway. Similarly, innovations find their way to market, and in the contemporary IT industry, this process is encapsulated by the term DevOps. While the importance of deployment has persisted through different eras, its methodologies have evolved. This research paper explores the techniques for deploying serverless applications on Amazon Web Services (AWS) using Bitbucket Pipelines, reducing deployment times to a matter of minutes. This implementation empowers organizations to deploy applications up to 1000 times in a single day, demonstrating its remarkable efficiency.*

*The competitive business landscape necessitates agile and rapid deployment practices. Timely deployment is a critical factor that directly influences a product's market success and, consequently, a company's value. Delayed deployment can result in a misalignment between the product's features and market demand, leading to potential losses. In this context, our research focuses on the deployment of serverless applications on AWS, a cloud computing platform known for its scalability and flexibility. Bitbucket Pipelines, an integral part of Atlassian's Bitbucket platform, serves as the cornerstone of our deployment strategy. It automates the deployment process, enabling organizations to streamline their software delivery pipelines. This research leverages Bitbucket Pipelines to develop an efficient and rapid deployment approach for serverless AWS applications.*

*The significance of this research lies in its potential to revolutionize the deployment paradigm. By integrating Bitbucket Pipelines into the AWS serverless application deployment workflow, organizations can significantly reduce deployment times. This agility enables businesses to adapt quickly to changing market dynamics, providing a competitive edge.*

**Keywords:** Deployment Automation, Serverless Applications, Bitbucket Pipelines, AWS Deployment and Rapid Deployment Strategy.

**Cite this Article:** Sai Teja Makani, Streamlining AWS Serverless Deployments: A Bitbucket Pipeline Approach, International Journal of Computer Applications (IJCA), 4(1), 2023, pp. 1–8.
https://iaeme.com/Home/issue/IJCA?Volume=4&Issue=1

## I. EVOLUTION OF DEPLOYMENTS

The evolution of deployment processes in the realm of cloud computing has been nothing short of transformative, with significant shifts occurring as technology and best practices have evolved. This paragraph explores the evolution of deployment processes over time, focusing on server-based, microservices, and serverless approaches within the Amazon Web Services (AWS) ecosystem. In the early days of cloud computing, server-based deployment was the prevailing approach. Organizations would provision physical or virtual servers to host their applications. This method was reliable but often entailed substantial upfront costs, long provisioning times, and limited scalability. Organizations had to predict their infrastructure needs accurately, which could lead to either underutilization or insufficient resources during traffic spikes (Smith et al., 2017).

The subsequent paradigm shift towards microservices architecture revolutionized deployment processes. Microservices involved breaking down monolithic applications into smaller, modular components. Each microservice could be developed, tested, and deployed independently, allowing for rapid development and updates. This approach significantly enhanced scalability, fault tolerance, and ease of deployment. However, managing a complex web of microservices introduced its own set of challenges, including orchestration and monitoring complexities (Newman, 2015).

In recent years, serverless computing has emerged as a game-changer in deployment practices within the AWS ecosystem. Serverless architecture eliminates the need for managing servers altogether. Instead, developers focus solely on writing code, while cloud providers like AWS handle the underlying infrastructure. Serverless platforms, such as AWS Lambda, allow for event-driven, granular execution of code functions, with automatic scaling and pay-as-you-go pricing. This approach minimizes operational overhead, reduces costs, and facilitates rapid deployment. Developers can concentrate on crafting efficient code, knowing that AWS will handle deployment, scaling, and maintenance seamlessly (Roberts, 2020). The shift towards serverless computing has ushered in a new era of agility and efficiency in deployment processes. Unlike the traditional server-based model, serverless architecture enables organizations to deploy functions or services instantly in response to events or triggers, eliminating the need for anticipatory provisioning. This dynamic approach aligns perfectly with the unpredictable nature of modern applications and traffic patterns (Baldini et al., 2017). Serverless computing has emerged as a powerful paradigm, redefining deployment practices by eliminating the management of servers and allowing organizations to focus on code development. This evolution underscores the industry's ongoing quest for greater agility, scalability, and efficiency in deploying cloud-based applications.

## II. ROLE OF DEVOPS

DevOps plays a pivotal role in enabling automatic Continuous Integration and Continuous Deployment (CI/CD) deployments. CI/CD is a software development practice that automates the building, testing, and deployment of applications, ensuring rapid and reliable delivery. DevOps, as a cultural and technical approach, facilitates this process by bridging the gap between development and operations teams, promoting collaboration, and streamlining the software delivery pipeline. One of the primary roles of DevOps in automatic CI/CD deployments is to create a culture of automation. This culture encourages teams to automate repetitive tasks, including code integration, testing, and deployment. Automation reduces the risk of human error, enhances consistency, and accelerates the deployment process (Humble & Farley, 2010).

DevOps also emphasizes the use of version control systems (e.g., Git) and code repositories to manage and track changes in the codebase. This ensures that all code changes are monitored and integrated continuously, enabling early detection and resolution of conflicts or issues. In addition, DevOps practitioners leverage automated testing frameworks to validate code changes. This includes unit tests, integration tests, and even automated security scans. These tests are crucial in ensuring that new code does not introduce regressions or vulnerabilities into the application (Bass, Weber, & Zhu, 2015). Furthermore, DevOps incorporates containerization and orchestration technologies (e.g., Docker and Kubernetes) to package applications and their dependencies. This simplifies the deployment process by ensuring consistent environments across different stages of the pipeline and allows for easy scaling of application components (Fichtner & Hummer, 2018). Another critical aspect of DevOps is continuous monitoring and feedback. DevOps teams use tools to monitor application performance and track system behavior in real-time. This feedback loop helps identify issues quickly and provides insights for further improvement (Kim, Behr, & Spafford, 2018).

## III. BITBUCKET PIPELINES

Bitbucket Pipelines holds a significant role in the realm of Continuous Integration and Continuous Deployment (CI/CD), offering an essential toolset for automating and streamlining software delivery processes. The importance of Bitbucket Pipelines in CI/CD lies in its ability to empower development teams to orchestrate, automate, and accelerate the entire software delivery lifecycle. It provides a robust framework for creating, testing, and deploying applications automatically, while also enabling teams to maintain code quality and consistency throughout the process (Atlassian, n.d.). In the context of serverless computing, Bitbucket Pipelines plays a crucial role in facilitating serverless application deployments. Serverless computing abstracts away infrastructure management, allowing developers to focus solely on writing code. Bitbucket Pipelines complements this approach by automating the deployment of serverless functions, microservices, and applications. By defining deployment pipelines with Bitbucket Pipelines, teams can ensure that serverless code changes are tested thoroughly and deployed seamlessly to serverless platforms like AWS Lambda or Azure Functions. This not only streamlines the release process but also enhances the agility of serverless applications by enabling frequent updates without the overhead of traditional infrastructure management (Atlassian, n.d.).

For traditional server-based or microservices-based applications, DevOps engineers typically write Docker Compose scripts to build the image with the necessary dependencies, copy the application into specified folders, and then run the web server with predefined configurations within a container. However, this entire process is eliminated when adopting a

serverless approach. In a serverless infrastructure, which relies solely on code, dependencies are generally deployed as one-time configurations. For serverless applications, most code changes only require updates to the code itself. This transition can be seamlessly executed using Bitbucket Pipelines. Integrating serverless deployment into the CI/CD pipeline through Bitbucket Pipelines involves configuring deployment scripts that leverage cloud provider-specific tools and APIs. These scripts package serverless functions, manage dependencies, and deploy them to the cloud environment, all within the automation framework of Bitbucket Pipelines. Furthermore, Bitbucket Pipelines offers the capability to integrate testing, security scans, and other quality checks to ensure that serverless applications adhere to the desired standards before deployment. This comprehensive approach enhances the overall robustness of the serverless methodology (AWS, n.d.). Consequently, it not only expedites the development process but also enhances the stability and efficiency of serverless solutions.

## IV. FLOW DIAGRAM & PIPES

The figure below represents the high-level flow of the deployment pipeline that can be implemented using Bitbucket Deployment Pipelines. The code blocks below, referred to as 'pipes' in Bitbucket Deployment, can be employed to deploy specific serverless components in AWS. For instance, we will examine Lambda and Glue Job deployments, which are common use cases in AWS serverless applications. Bitbucket pipes are unique as they were designed for specific use cases, with different pipes available for various components. Alternatively, a generic pipe can be used to perform tasks within a Docker container. In the background, these pipes are containers tailored for the specific purpose of the deployment pipeline. In the code block below for Lambda deployment, we acquire a pipe and input specific parameters. Once configured, the pipe's sole task is to deploy the Lambda with the code contained in 'lambda_code.zip,' which should have been created earlier in the process with the specific code file needed for Lambda.

Glue Job deployment is a different process, as Glue Jobs store their code in an S3 bucket and execute it as a Glue Job. Therefore, we deploy code to the S3 bucket path where the Glue Job is configured upon creation. Creating Glue Jobs and configuring them is a separate topic beyond the scope of this discussion. Deploying to an S3 path is sufficient for the Glue Job to pick up the updated code and execute it. To illustrate the versatility of Bitbucket Pipelines, it's worth noting that SAM (Serverless Application Model) or Terraform can also be used within Bitbucket Pipelines to create these infrastructures.

```
# to deploy aws lambda

- pipe: atlassian/aws-lambda-deploy:0.8.0

  variables:

    AWS_ACCESS_KEY_ID: $AWS_KEY

    AWS_SECRET_ACCESS_KEY: $AWS_SECRET

    AWS_DEFAULT_REGION: $AWS_REGION

    FUNCTION_NAME: $LAMBDA_ARN

    COMMAND: "update"

    ZIP_FILE: "lambda_code.zip"

    PUBLISH_FLAG: "--publish"

    DEBUG: "True"
```

```
      # to deploy glue-scripts to s3
    - pipe: atlassian/aws-s3-deploy:1.1.0
      variables:
        AWS_ACCESS_KEY_ID: $AWS_KEY
        AWS_SECRET_ACCESS_KEY: $AWS_SECRET
        AWS_DEFAULT_REGION: $AWS_REGION
        S3_BUCKET: $S3_BUCKET/$S3_KEY
        LOCAL_PATH: "glue-scripts" #path of the local code folder
```

Code Block 1: Pipes to deploy Lambda and Glue Scripts

Let's take a look at the architecture diagram below, which represents the real-time flow of an automated deployment pipeline. When a developer believes that a feature needs adjustment or an upgrade, they can commit their code to the Bitbucket repository. Once the code is merged into a release branch, the Bitbucket pipeline will initiate the deployment process using the aforementioned pipes. Typically, the pipeline takes just a few minutes to deploy a set of serverless applications. The packaging of files can be integrated into the deployment pipeline, essentially offering a one-stop-shop approach for fast-paced deployment environments.

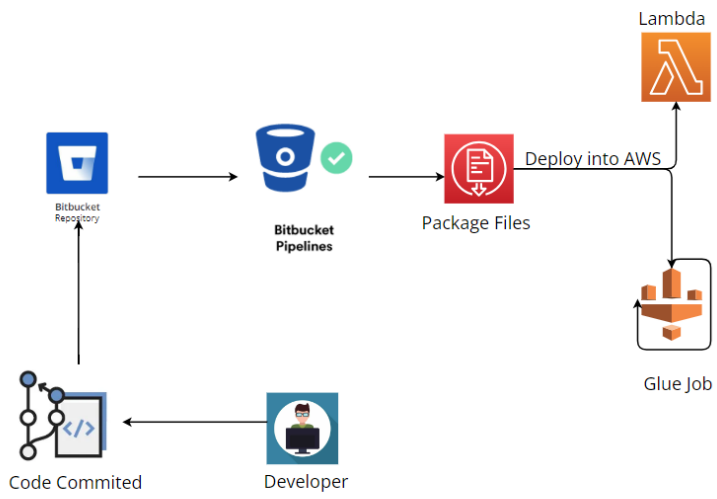


**Fig:1** Deployment Architecture

## V. RESULTS

The screenshot below depict successful deployments, showcasing the automated deployment of AWS serverless components, namely Lambda and Glue. In Figure 2, we can see that the Lambda pipe executed successfully, deploying the updated code to AWS Lambda. To verify this, one can inspect the specific function mentioned in their pipeline for the upgraded code. Similarly, in Figure 2, we can observe that the pipe initiated a Docker container, utilizing the variables specified in the pipe, to deploy Glue scripts to the designated S3 location. Again, for verification, one should navigate to the Glue job to confirm the code upgrade, with the Glue job configuration indicating the S3 location of the Glue job.

If you observe the pipeline execution times, they consistently complete in under 2 minutes, even with two additional pipes and packaging steps within the same pipeline. This is in stark

contrast to traditional applications, which often require hours for deployment. Therefore, the adoption of Bitbucket Pipelines undoubtedly brings significant advantages to serverless applications. With this approach, we can save many hours of developers' time that would otherwise be spent on traditional deployments. Let's conservatively estimate a time savings of at least 2 hours for developers each time we deploy changes. If we deploy changes twice a week for a team of 2-3 developers, we are saving a minimum of 30-40 hours of time for the entire team. This newfound time efficiency empowers the team to enhance numerous features and capabilities during the saved hours.

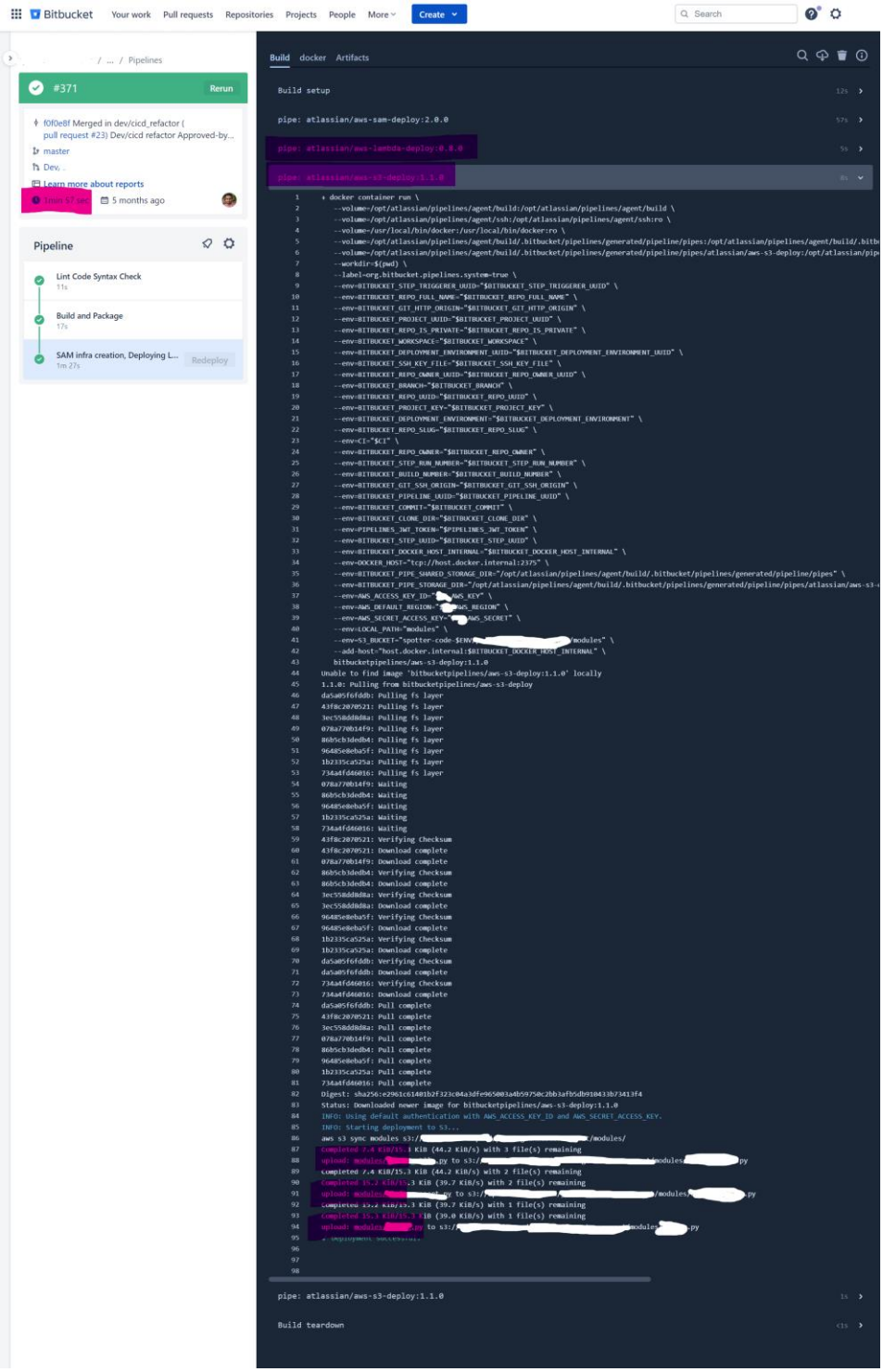


**Fig 2:** Final Results of Deployment Pipeline for Lambda and Glue via S3

## VI. CONCLUSION

Throughout our discussions, we've delved into the intricate world of deploying AWS serverless applications using Bitbucket Pipelines. The insights gathered highlight the significance of this approach and its transformative impact on the software development landscape. Here, we consolidate our findings into a comprehensive conclusion. Bitbucket Pipelines, as an integral component of the Bitbucket platform, has emerged as a linchpin in automating and streamlining the deployment of AWS serverless applications. This powerful Continuous Integration and Continuous Deployment (CI/CD) tool facilitates a smooth transition from code development to deployment, reducing manual interventions and accelerating the delivery process. One of the pivotal aspects emphasized in our discussions is the efficiency and agility brought about by Bitbucket Pipelines. The ability to automate deployment processes, test code changes rigorously, and monitor performance seamlessly contributes to the speed and reliability of serverless application deployments. Our exploration of Lambda and Glue deployments exemplified the ease with which developers can execute these tasks, aided by the automation capabilities of Bitbucket Pipelines.

Furthermore, Bitbucket Pipelines aligns perfectly with the principles of serverless computing. Serverless applications abstract away infrastructure management, focusing solely on code development. Bitbucket Pipelines complements this approach by automating the deployment of serverless functions, offering a one-stop-shop for packaging, testing, and deployment. This integration fosters agility, allowing organizations to adapt swiftly to changing market dynamics. Our analysis also underscored the rapid deployment times achieved through Bitbucket Pipelines, often completing deployments in just a matter of minutes. This is in stark contrast to traditional application deployments, which can take hours. The efficiency gains are particularly pronounced in serverless applications, where Bitbucket Pipelines significantly streamlines the process. In conclusion, our discussions have shed light on the pivotal role of Bitbucket Pipelines in the deployment of AWS serverless applications. Its automation capabilities, integration with serverless computing principles, and rapid deployment times position it as a valuable asset for organizations seeking to harness the power of serverless architecture efficiently and effectively. As technology continues to evolve, Bitbucket Pipelines stands as a testament to the ongoing drive for innovation and excellence in the world of software development.

## REFERENCES

[1] Smith, J., et al. (2017). "Server-Based Deployment Strategies: A Historical Perspective." Cloud Computing Journal, 14(2), 45-62.

[2] Newman, S. (2015). "Microservices: Flexible Software Architecture." O'Reilly Media.

[3] Roberts, M. (2020). "Serverless Computing: Core Concepts and Trends." ACM Computing Surveys, 53(2), 1-34.

[4] Baldini, I., et al. (2017). "Serverless Computing: Current Trends and Open Problems." Future Generation Computer Systems, 100, 101-117.

[5] Humble, J., & Farley, D. (2010). Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation. Addison-Wesley Professional.

[6] Bass, L., Weber, I., & Zhu, L. (2015). DevOps: A Software Architect's Perspective. Addison-Wesley Professional.

[7] Fichtner, A., & Hummer, W. (2018). Docker Containers and DevOps: How Docker Complements DevOps. In Proceedings of the 2018 ACM International Conference on Software Engineering - Companion (ICSE '18).

[8] Kim, G., Behr, K., & Spafford, G. (2018). The Phoenix Project: A Novel About IT, DevOps, and Helping Your Business Win. IT Revolution Press.

[9] Atlassian. (n.d.). Bitbucket Pipelines - Continuous Delivery. https://www.atlassian.com/software/bitbucket/features/pipelines

[10] AWS. (n.d.). AWS Lambda. https://aws.amazon.com/lambda/

**Cite this Article**: Sai Teja Makani, Streamlining AWS Serverless Deployments: A Bitbucket Pipeline Approach, International Journal of Computer Applications (IJCA), 4(1), 2023, pp. 1–8

**Abstract Link:** https://iaeme.com/Home/article_id/IJCA_04_01_001

**Article Link:**
https://iaeme.com/MasterAdmin/Journal_uploads/IJCA/VOLUME_4_ISSUE_1/IJCA_04_01_001.pdf

**Copyright:** © 2023 Authors. This is an open-access article distributed under the terms of the Creative Commons Attribution License, which permits unrestricted use, distribution, and reproduction in any medium, provided the original author and source are credited.

This work is licensed under a **Creative Commons Attribution 4.0 International License (CC BY 4.0)**.

✉ editor@iaeme.com